Assignment 5

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question

Let $S = \{0, 1, 2, \ldots, n-1\}$ be a set of n distinct elements. Initially, each element forms its own separate set, so the collection consists of n disjoint singleton sets. This initial configuration is represented using a parent array P of size n, where $P[i] = i$ for all $0 \leq i \leq n-1$, and a rank array R, where $R[i] = 0$ for all $0 \leq i \leq n-1$.

Design and implement an efficient Disjoint Set (Union-Find) data structure that supports two fundamental operations: Find(x), which returns the representative (root) of the set containing element x, and Union(x, y), which merges the two sets containing elements x and y.

To improve efficiency, apply path compression during the Find operation so that each visited node directly points to the root, and apply union by rank during the Union operation so that the tree with smaller height is attached under the tree with larger height, thereby keeping the tree shallow.

Code

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *P; //parent array of size n
    int *R; //rank array of size n
    int n;
} DisjointSet;

DisjointSet* createSet(int n) {
    DisjointSet* ds = (DisjointSet*)malloc(sizeof(DisjointSet));
    ds->n = n;
    ds->P = (int*)malloc(n * sizeof(int));
    ds->R = (int*)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        ds->P[i] = i;
        ds->R[i] = 0;
    }
    return ds;
}

// Find with Path Compression
int find(DisjointSet* ds, int x) {
```

```c
    if (ds->P[x] != x) {
        ds->P[x] = find(ds, ds->P[x]); // Path compression
    }
    return ds->P[x];
}

// Union by Rank
void unionSets(DisjointSet* ds, int x, int y) {
    int rootX = find(ds, x);
    int rootY = find(ds, y);

    if (rootX != rootY) {
        // Attach smaller rank tree under higher rank tree
        if (ds->R[rootX] > ds->R[rootY]) {
            ds->P[rootY] = rootX;
        } else if (ds->R[rootX] < ds->R[rootY]) {
            ds->P[rootX] = rootY;
        } else {
            ds->P[rootY] = rootX;
            ds->R[rootX]++; // Increase rank if heights are the same
        }
        printf("Union successful: %d and %d merged.\n", x, y);
    } else {
        printf("Elements %d and %d are already in the same set.\n", x, y);
    }
}

// Free memory
void freeSet(DisjointSet* ds) {
    free(ds->P);
    free(ds->R);
    free(ds);
}

int main() {
    int n;

    printf("Enter the total number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input. Number of elements must be greater than 0.\n");
        return 1;
    }

    DisjointSet* ds = createSet(n);
    int choice, x, y;

    while (1) {
        printf("\n--- Disjoint Set Operations ---\n");
        printf("1. Union two elements (x, y)\n");
```

```c
        printf("2. Find the representative of an element (x)\n");
        printf("3. Exit\n");
        printf("Enter your choice (1/2/3): ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input.\n");
            break;
        }

        switch (choice) {
            case 1:
                printf("Enter two elements to union (separated by space): ");
                scanf("%d %d", &x, &y);
                if (x >= 0 && x < n && y >= 0 && y < n) {
                    unionSets(ds, x, y);
                } else {
                    printf("Error: Elements must be between 0 and %d.\n", n - 1);
                }
                break;

            case 2:
                printf("Enter element to find: ");
                scanf("%d", &x);
                if (x >= 0 && x < n) {
                    printf("The representative (root) of %d is: %d\n", x, find(ds, x));
                } else {
                    printf("Error: Element must be between 0 and %d.\n", n - 1);
                }
                break;

            case 3:
                printf("Exiting and cleaning up memory...\n");
                freeSet(ds);
                return 0;

            default:
                printf("Invalid choice. Please select 1, 2, or 3.\n");
        }
    }

    freeSet(ds);
    return 0;
}
```

Output

```
Enter the total number of elements: 3

--- Disjoint Set Operations ---
1. Union two elements (x, y)
2. Find the representative of an element (x)
3. Exit
Enter your choice (1/2/3): 1
Enter two elements to union (separated by space): 0 1
Union successful: 0 and 1 merged.

--- Disjoint Set Operations ---
1. Union two elements (x, y)
2. Find the representative of an element (x)
3. Exit
Enter your choice (1/2/3): 1
Enter two elements to union (separated by space): 1 2
Union successful: 1 and 2 merged.

--- Disjoint Set Operations ---
1. Union two elements (x, y)
2. Find the representative of an element (x)
3. Exit
Enter your choice (1/2/3): 1
Enter two elements to union (separated by space): 2 3
Error: Elements must be between 0 and 2.

--- Disjoint Set Operations ---
1. Union two elements (x, y)
2. Find the representative of an element (x)
3. Exit
Enter your choice (1/2/3): 2
Enter element to find: 2
The representative (root) of 2 is: 0

--- Disjoint Set Operations ---
1. Union two elements (x, y)
2. Find the representative of an element (x)
3. Exit
Enter your choice (1/2/3): 3
Exiting and cleaning up memory...
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5>
```

Figure 1: Output for Problem 1.

Complexity Analysis

Time Complexity:

- **Initialization:** $O(n)$ to create the parent and rank arrays.
- **Find and Union Operations:** Because we use both path compression and union by rank, the trees stay extremely flat. This makes both operations take nearly constant time, or $O(1)$ on average.

Space Complexity: $O(n)$ to store the parent and rank arrays for the n elements.

Problem 2

Question

In a large school campus, thousands of students dynamically form different friend groups, where new friendships are created daily and teachers frequently need to verify whether two students belong to the same group. Initially, each student forms an independent group. As interactions increase, multiple groups gradually merge, and it becomes essential to efficiently manage these dynamic group connections.

Given n students and m randomly generated friendship operations, design and implement an optimized Disjoint Set (Union-Find) data structure using dynamic arrays, which supports two fundamental operations:

- Union(x, y): Merges the sets containing students x and y.
- Find(x): Returns the representative (leader) of the set containing student x.

Let the initial parent array be defined as:

$$P[i] = i, \quad \text{for all } 0 \leq i \leq n-1$$

To enhance performance, apply path compression during the Find operation and union by rank during the Union operation. The system must efficiently process all m operations, ensuring fast connectivity queries even for large values of n and m.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int *P;
int *R;

void initStudents(int n) {
    P = (int*)malloc(n * sizeof(int));
    R = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        P[i] = i;
        R[i] = 0;
    }
}

int findStudent(int x) {
    if (P[x] != x) {
        P[x] = findStudent(P[x]);
    }
    return P[x];
}

void unionStudents(int x, int y) {
```

```c
    int rootX = findStudent(x);
    int rootY = findStudent(y);

    if (rootX != rootY) {
        if (R[rootX] > R[rootY]) {
            P[rootY] = rootX;
        } else if (R[rootX] < R[rootY]) {
            P[rootX] = rootY;
        } else {
            P[rootY] = rootX;
            R[rootX]++;
        }
        printf("Students %d and %d became friends!\n", x, y);
    } else {
        printf("Students %d and %d are already in the same friend group.\n", x, y);
    }
}

int main() {
    int n, m;
    printf("Enter number of students (n): ");
    scanf("%d", &n);
    printf("Enter number of random friendship operations (m): ");
    scanf("%d", &m);

    initStudents(n);
    srand(time(NULL));

    printf("\n--- Processing %d Random Friendships ---\n", m);
    for (int i = 0; i < m; i++) {
        int u = rand() % n;
        int v = rand() % n;
        unionStudents(u, v);
    }

    free(P);
    free(R);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5> gcc Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5> ./a.exe
Enter number of students (n): 15
Enter number of random friendship operations (m): 20

--- Processing 20 Random Friendships ---
Students 13 and 13 are already in the same friend group.
Students 3 and 14 became friends!
Students 8 and 6 became friends!
Students 12 and 13 became friends!
Students 14 and 10 became friends!
Students 10 and 10 are already in the same friend group.
Students 8 and 5 became friends!
Students 7 and 0 became friends!
Students 9 and 3 became friends!
Students 4 and 2 became friends!
Students 2 and 11 became friends!
Students 3 and 4 became friends!
Students 4 and 12 became friends!
Students 10 and 1 became friends!
Students 8 and 8 are already in the same friend group.
Students 8 and 13 became friends!
Students 6 and 2 are already in the same friend group.
Students 11 and 7 became friends!
Students 7 and 8 are already in the same friend group.
Students 3 and 9 are already in the same friend group.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5>
```

Figure 2: Output for Problem 2.

Complexity Analysis

Time Complexity:

- **Initialization:** $O(n)$ to set up the independent student groups.
- **Friendship Operations:** We process m random operations. Since each `Union` and `Find` takes nearly constant time ($O(1)$) due to our optimizations, processing all m friendships takes $O(m)$ time.
- **Total Time Complexity:** $O(n + m)$, making it highly efficient even for thousands of students.

Space Complexity: $O(n)$ for the dynamically allocated arrays storing the student connections.

Problem 3

Question

A city contains N buildings, initially all isolated, which can be mathematically modeled as N disjoint singleton sets, where the parent array is initialized as:

$$P[i] = i, \quad \text{for all } 0 \leq i \leq N-1.[cite_start]$$

Over time, M road construction operations are performed, where each operation connects two buildings (u, v), with $0 \leq u, v \leq N-1$. To efficiently manage these dynamic connections, design a Disjoint Set (Union-Find) data structure supporting two operations: Union(u, v), which merges the sets containing u and v, and Find(x), which returns the representative (leader) of the set containing x.

After each union operation, the system must verify whether the entire city has become fully connected, which mathematically implies that all buildings belong to a single connected component, i.e., there exists a unique representative r such that:

$$Find(i) = r \quad \text{for all } 0 \leq i \leq N-1$$

Code

```c
#include <stdio.h>
#include <stdlib.h>

int *P;
int *R;
int components;

void initCity(int N) {
    P = (int*)malloc(N * sizeof(int));
    R = (int*)malloc(N * sizeof(int));
    components = N;

    for (int i = 0; i < N; i++) {
        P[i] = i;
        R[i] = 0;
    }
}

int findBuilding(int x) {
    if (P[x] != x) {
        P[x] = findBuilding(P[x]);
    }
    return P[x];
}

void buildRoad(int u, int v) {
    int rootU = findBuilding(u);
    int rootV = findBuilding(v);
```

```c

    if (rootU != rootV) {
        if (R[rootU] > R[rootV]) {
            P[rootV] = rootU;
        } else if (R[rootU] < R[rootV]) {
            P[rootU] = rootV;
        } else {
            P[rootV] = rootU;
            R[rootU]++;
        }
        components--;
        printf("Road built between %d and %d.\n", u, v);
    } else {
        printf("Buildings %d and %d are already connected.\n", u, v);
    }
}

int main() {
    int N, M;
    printf("Enter number of buildings (N): ");
    scanf("%d", &N);

    initCity(N);

    printf("Enter number of road constructions (M): ");
    scanf("%d", &M);

    printf("\nEnter the %d road connections (u v):\n", M);
    for (int i = 0; i < M; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        buildRoad(u, v);

        if (components == 1) {
            printf("The entire city is now FULLY connected!\n");
        } else {
            printf("City has %d separate components remaining.\n",
   components);
        }
    }

    free(P);
    free(R);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5> gcc .\Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5> ./a.exe
Enter number of buildings (N): 5
Enter number of road constructions (M): 4

Enter the 4 road connections (u v):
0 1
Road built between 0 and 1.
City has 4 separate components remaining.
1 2
Road built between 1 and 2.
City has 3 separate components remaining.
2 3
Road built between 2 and 3.
City has 2 separate components remaining.
3 4
Road built between 3 and 4.
The entire city is now FULLY connected!
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment5>
```

Figure 3: Output for Problem 3.

Complexity Analysis

Time Complexity:

- **Initialization:** $O(N)$ to set up the isolated buildings.
- **Road Construction:** Processing M roads takes $O(M)$ time since each optimized `Union` takes practically $O(1)$ time.
- **Checking Connectivity:** Checking if the whole city is connected takes strictly $O(1)$ time because we just check if our `components` variable equals 1.
- **Total Time Complexity:** $O(N + M)$.

Space Complexity: $O(N)$ to store the data structures for the city buildings.